Integração e Entrega Contínua – DSM – Professora Lucineide – 1ºSemestre/2026

Atividade Prática – Aula 7

Tema – Introdução a Testes Automatizados Avançados

Entrega da Atividade

1. Exporte/salve o arquivo de acordo com o padrão: **IEC-Atividade-Aula07.pdf**
2. Enviar a Atividade em PDF no GitHub:
    1. Acesse seu repositório da disciplina: **FATEC-JCR-4DSM-IEC-2026-1-seunome**
    2. Se ainda não existir, você deverá criar a pasta da atividade: **P2-Conteudos/Atividades/Atividade-Aula07/README.md**
    3. Faça o **commit** do pdf da atividade feita na pasta.
    4. Em seguida copie o **link** do pdf.
    5. Acesse o **Kanban-IEC-2026-1** da disciplina que se encontra no repositório da professora.
    6. Clique no card: **IEC-Atividade-Aula07**.
    7. No comentário, cole o **link** do **pdf** da atividade que você fez.
    8. Volte ao seu repositório e acesse o seu **Kanban-IEC-2026-1.**
    9. Mova o card **IEC-Atividade-Aula07** para a coluna **Entregue**.
3. Ou, no caso de configurações feitas no repositório do GitHub, enviar o link.

Exercício 1

Criar teste de integração completo para validar a execução conjunta de funções.

Exemplo: crie um arquivo processamento.test.js.

Adicione o teste:

```
test("processamento completo de alerta crítico", () => {
const resultado = processarAlerta(90);
expect(resultado).toBe("Notificação enviada: Crítico");
});
```

Execute npm test.

Confirme que o teste passa.

Exercício 2

Crie mock de função externa para simular chamadas externas com Jest.

Exemplo:

Crie um mock:

const api = { enviar: jest.fn(() => "Simulado!") };

Teste o mock:

test("simulação de envio", () => {

const resposta = api.enviar();

expect(resposta).toBe("Simulado!");

});

Exercício 3

Execute verificação de segurança para garantir que as dependências estão seguras.

Exemplo:

No terminal, rode:

npm audit

Veja se há alertas de vulnerabilidade.

Atualize pacotes inseguros com:

npm audit fix

Exercício 4

Adicione workflow de segurança para automatizar a auditoria de dependências.

Exemplo:

Crie *.github/workflows/security.yml*.

Cole o conteúdo do exemplo acima.

Faça *commit* no *dev*.

Observe a execução no GitHub Actions.

Exercício 5

Fça um commit final e um PR para consolidar os testes e auditorias no pipeline.

Exemplo:

Crie uma *branch feature/seguranca*.

Faça *commit* dos arquivos novos.

Abra PR no *dev*.

Verifique se o pipeline executa com sucesso (lint + testes + auditoria).

Exercício 6

Crie o teste unitário como boa prática para fortalecer a prática de testes isolados.

Exemplo: na pasta /tests, crie um arquivo alerta.test.js.

Implemente o teste:

```
test("alerta alto", () => {
expect(classificarAlerta(70)).toBe("Alto");
});
```

Rode os testes com: *npm test*

Exercício 7

Crie o teste de integração como boa prática para validar se funções combinadas funcionam corretamente.

Exemplo: crie um arquivo integracao.test.js.

Escreva:

```
test("classificação + notificação", () => {
const alerta = classificarAlerta(90);
const resultado = enviarNotificacao(alerta);
expect(resultado).toBe("Notificação enviada: Crítico");
});
```

Execute *npm test* e valide o resultado.

Exercício 8

Simule um erro em teste para que se entenda que as falhas são tão importantes quanto acertos. Exemplo: no teste unitário, troque o valor esperado:

expect(classificarAlerta(90)).toBe("Alto");

Rode *npm test*.

Observe o erro e corrija para o valor correto.

Exercício 9

Faça com que seja gerado um relatório de cobertura para medir a qualidade dos testes. Exemplo, rode:

npm test -- --coverage

Observe a porcentagem de cobertura.

Tente aumentar criando testes.

Exercício 10

Crie Commit e PR com novos testes para integrar ao pipeline.

Exemplo:

Faça commit dos novos testes.

Abra Pull Request de *feature/testes no dev*.

Acompanhe o pipeline no GitHub Actions (lint e testes).

Corrija se houver falhas.